

LINUX ADMINISTRATION — COMMAND NOTES

1. date

Definition

The date command displays or sets the system date and time. It is commonly used to check the current date/time and format it for scripts or logs.

Syntax

```
date
```

```
date [OPTIONS]
```

```
date +FORMAT
```

Examples

```
date
```

```
date -u
```

```
date +%Y-%m-%d
```

```
date +%H:%M:%S
```

Example Output

```
Thu Sep 24 20:15:30 IST 2026
```

Explanation

- date → Displays the current local date and time.
- date -u → Displays UTC time.
- +%Y-%m-%d → Displays year-month-day.
- +%H:%M:%S → Displays hour:minute:second.

Common format characters:

Format	Meaning
%Y	4-digit year
%m	Month
%d	Day
%H	Hour (24-hour)
%M	Minute
%S	Second

Use Cases

- Checking system time.
- Generating timestamps.
- Using dates in shell scripts.
- Naming backup files with timestamps.

2. uptime

Definition

The uptime command displays how long the system has been running, the number of logged-in users, and system load averages.

Syntax

```
uptime  
uptime [OPTIONS]
```

Examples

```
uptime  
uptime -p  
uptime -s
```

Example Output

```
20:20:10 up 5 days, 3:21, 2 users, load average: 0.20, 0.15,  
0.10
```

Explanation

- uptime → Shows current time, uptime, users, and load averages.
- uptime -p → Shows uptime in a human-readable form.
- uptime -s → Shows the date/time when the system was booted.

The load averages represent approximately:

1 minute 5 minutes 15 minutes

Use Cases

- Checking system availability.
- Checking system load.
- Troubleshooting performance problems.
- Finding when the system was booted.

3. uname

Definition

The uname command displays information about the operating system and kernel.

Syntax

```
uname  
uname [OPTION]
```

Examples

```
uname  
uname -a  
uname -r  
uname -m  
uname -n
```

Example Output

```
Linux
```

Explanation

Important options:

Option	Meaning
-s	Kernel name
-n	Network hostname

-r	Kernel release
-v	Kernel version
-m	Machine hardware architecture
-p	Processor type
-i	Hardware platform
-o	Operating system
-a	All available information

4. `uname -a`

Definition

`uname -a` displays all available system information provided by `uname`.

Syntax

```
uname -a
```

Example

```
uname -a
```

Example Output

```
Linux server01 6.8.0-40-generic #40-Ubuntu SMP x86_64 GNU/
Linux
```

Explanation

It generally provides:

- Kernel name
- Hostname
- Kernel release
- Kernel version
- Machine architecture
- Processor/platform information
- Operating system

Use Cases

- Identifying the Linux kernel.
- Checking architecture.
- Troubleshooting compatibility issues.
- Collecting system information.

5. `cal`

Definition

The `cal` command displays a calendar in the terminal.

Syntax

```
cal
```

```
cal YEAR
```

```
cal MONTH YEAR
```

Examples

```
cal
```

```
cal 2026
```

```
cal 9 2026
```

Explanation

- `cal` → Displays the current month's calendar.
- `cal 2026` → Displays the calendar for the year 2026.
- `cal 9 2026` → Displays September 2026.

Use Cases

- Checking dates.
- Quickly viewing a monthly/yearly calendar.
- Shell-based date reference.

6. `bc`

Definition

`bc` is a command-line calculator that supports arithmetic and more advanced mathematical operations.

Syntax

```
bc  
echo "expression" | bc
```

Examples

```
bc  
echo "10 + 20" | bc  
echo "10 / 3" | bc  
echo "scale=2; 10 / 3" | bc
```

Example Output

```
3.33
```

Explanation

`scale` specifies the number of digits after the decimal point.

For example:

```
echo "scale=2; 10 / 3" | bc
```

produces:

```
3.33
```

Use Cases

- Command-line calculations.
- Shell scripts.
- Floating-point calculations.

7. `systemctl`

Definition

`systemctl` is used to manage services and other systemd units on Linux systems using `systemd`.

Syntax

```
systemctl COMMAND SERVICE
```

A service can be specified as:

```
service
```

or:

```
service.service
```

Start a Service

```
sudo systemctl start sshd
```

or:

```
sudo systemctl start sshd.service
```

Starts the service.

Stop a Service

```
sudo systemctl stop sshd
```

Stops the service.

Check Status

```
systemctl status sshd
```

Displays whether the service is running and recent service information.

Restart

```
sudo systemctl restart sshd
```

Stops and starts the service again.

Reload

```
sudo systemctl reload sshd
```

Reloads the service configuration without completely stopping the service, if that service supports reload.

Essential Variations

```
systemctl is-active sshd
```

```
systemctl is-enabled sshd
```

```
sudo systemctl enable sshd
```

```
sudo systemctl disable sshd
```

```
sudo systemctl enable --now sshd
```

Difference

- start → Start now.
- stop → Stop now.
- restart → Restart now.
- reload → Reload configuration.
- enable → Start automatically during boot.
- disable → Do not start automatically during boot.
- enable --now → Enable at boot and start immediately.

List Services

```
systemctl list-units --type=service
```

8. ps

Definition

ps displays information about currently running processes.

Basic Syntax

```
ps
```

Example

ps

Example Output

```
PID TTY          TIME CMD
1200 pts/0      00:00:00 bash
1250 pts/0      00:00:00 ps
```

9. ps -e

Definition

Displays all running processes.

Syntax

ps -e

Example

ps -e

Explanation

-e selects every process.

10. ps -ef

Definition

Displays all processes using full-format listing.

Syntax

ps -ef

Example

ps -ef

Important Columns

Column	Meaning
UID	User
PID	Process ID
PPID	Parent Process ID
C	CPU utilization
STIME	Start time
TTY	Terminal
TIME	CPU time
CMD	Command

Use Cases

- Finding processes.
- Finding parent-child relationships.
- Troubleshooting services.

11. ps aux

Definition

Displays detailed information about processes using BSD-style options.

Syntax

```
ps aux
```

Important Columns

Column	Meaning
USER	Process owner
PID	Process ID
%CPU	CPU usage
%MEM	Memory usage
VSZ	Virtual memory
RSS	Resident memory
STAT	Process state
START	Start time
TIME	CPU time
COMMAND	Command

Example

```
ps aux
```

12. ps -u username

Definition

Displays processes belonging to a specific user.

Syntax

```
ps -u USERNAME
```

Example

```
ps -u john
```

Use Case

Useful for finding all processes owned by a particular user.

13. Additional ps Variations

Process by PID

```
ps -p 1234
```

Detailed information for a PID

```
ps -f -p 1234
```

Search for a process

```
ps -ef | grep ssh
```

An alternative is:

```
pgrep ssh
```

14. top

Definition

top provides a real-time, continuously updating view of running processes and system resource usage.

Syntax

top

Example

top

Important Information

top can display:

- CPU usage
- Memory usage
- Load average
- Running processes
- Process IDs
- Process owners
- Process states

15. top -u username

Definition

Displays processes belonging to a specific user in top.

Syntax

top -u USERNAME

Example

top -u john

16. Important top Interactive Commands

Start:

top

Then use the following keys.

Press c

c

Toggles between displaying the process name and the full command line.

Press k

k

Allows you to send a signal to a selected process.

Typical workflow:

k

PID

15

This sends SIGTERM.

Press m

M

Sorts processes by memory usage.

Press p

P

Sorts processes by CPU usage.

Press q

q

Quits top.

17. kill

Definition

The kill command sends a signal to a process.

Despite its name, kill does not necessarily terminate a process. It can send many different signals.

Syntax

```
kill [SIGNAL] PID
```

or:

```
kill -SIGNAL PID
```

18. kill -1

Signal

SIGHUP

Example

```
kill -1 1234
```

Sends SIGHUP to PID 1234.

SIGHUP is commonly used to request that a daemon reload its configuration, although the exact behavior depends on the application.

19. kill -2

Signal

SIGINT

Example

```
kill -2 1234
```

Equivalent conceptually to an interrupt such as pressing Ctrl+C in a terminal.

20. kill -9

Signal

SIGKILL

Example

```
kill -9 1234
```

Immediately terminates the process.

Important

SIGKILL cannot be caught, blocked, or handled by the target process. Use it only when normal termination does not work.

21. kill -15

Signal

SIGTERM

Example

```
kill -15 1234
```

Requests that the process terminate gracefully.

This is generally preferable to SIGKILL when possible.

22. kill -l

Definition

Lists available signal names/numbers.

Syntax

```
kill -l
```

Example

```
kill -l
```

Common signals include:

Number	Signal	Purpose
1	SIGHUP	Hangup
2	SIGINT	Interrupt
9	SIGKILL	Force termination
15	SIGTERM	Graceful termination
19	SIGSTOP	Stop process
18	SIGCONT	Continue process

Signal numbers can vary on some architectures, so kill -l is the authoritative list for the current system.

23. Kill Using Signal Names

SIGINT

```
kill -SIGINT 1234
```

SIGKILL

```
kill -SIGKILL 1234
```

SIGTERM

```
kill -SIGTERM 1234
```

SIGSTOP

```
kill -SIGSTOP 1234
```

Stops/suspends the process.

SIGCONT

```
kill -SIGCONT 1234
```

Continues a stopped process.

24. id

Definition

The `id` command displays the user ID, primary group ID, and supplementary group memberships of a user.

Syntax

```
id
```

```
id USERNAME
```

Examples

```
id
```

```
id john
```

Example Output

```
uid=1000(john) gid=1000(john) groups=1000(john),27(sudo)
```

Explanation

- `uid` → User ID.
- `gid` → Primary Group ID.
- `groups` → Groups to which the user belongs.

Use Cases

- Checking user identity.
- Checking group membership.
- Troubleshooting file permissions.
- Verifying access permissions.

25. Shell Job Control — `jobs`, `bg`, `fg`

Definition

Shell job-control commands manage processes started from the current shell.

`jobs`

Displays jobs associated with the current shell.

```
jobs
```

`bg`

Continues a stopped job in the background.

```
bg %1
```

`fg`

Brings a job to the foreground.

```
fg
```

or:

```
fg %1
```

Typical Workflow

Start a command:

```
sleep 300
```

Press:

```
Ctrl+Z
```

Check jobs:

```
jobs
```

Continue in background:

`bg %1`

Bring it back to foreground:

`fg %1`

SIGCONT VS fg

`kill -SIGCONT <PID>`

resumes the process.

Whereas:

`fg %1`

brings a shell job into the foreground and the shell manages its job-control state.

26. crontab

Definition

crontab is used to schedule commands or scripts to run repeatedly at specified times.

Syntax

`crontab [OPTION]`

Edit Crontab

`crontab -e`

List Crontab

`crontab -l`

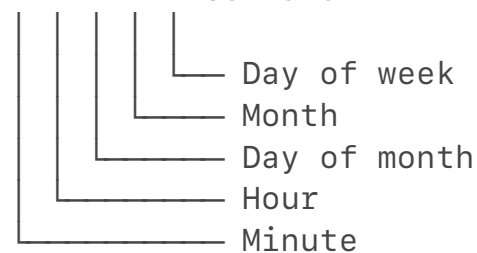
Remove Crontab

`crontab -r`

27. Cron Format

Syntax

`* * * * * command`



Example

`0 2 * * * /home/user/backup.sh`

This runs the script every day at 2:00 AM.

28. Cron Annotations

Cron provides special shortcuts beginning with @.

@reboot

`@reboot /home/user/start.sh`

Runs when the cron service starts/reboots according to cron's reboot behavior.

@yearly

```
@yearly /home/user/script.sh
```

Runs once a year.

```
@annually
```

```
@annually /home/user/script.sh
```

Equivalent to @yearly.

```
@monthly
```

```
@monthly /home/user/script.sh
```

Runs once a month.

```
@weekly
```

```
@weekly /home/user/script.sh
```

Runs once a week.

```
@daily
```

```
@daily /home/user/script.sh
```

Runs once a day.

```
@midnight
```

```
@midnight /home/user/script.sh
```

Runs approximately once a day around midnight.

```
@hourly
```

```
@hourly /home/user/script.sh
```

Runs once every hour.

29. at

Definition

at schedules a command or script to run **once at a specified future time**.

Unlike cron, it is intended for one-time jobs.

Syntax

```
at TIME
```

```
at now + NUMBER UNIT
```

30. at — Relative Time

Five minutes from now

```
at now + 5 minutes
```

Ten minutes from now

```
at now + 10 minutes
```

Two hours from now

```
at now + 2 hours
```

One day from now

```
at now + 1 day
```

You can then enter the command:

```
at now + 5 minutes
```

```
echo "Backup completed" > /tmp/backup.txt
```

Press:

Ctrl+D

to submit the job.

31. at — Specific Time

Examples:

`at 10:30`

`at 14:00`

`at 10:30 tomorrow`

Common natural-language forms supported by typical `at` implementations can include expressions such as:

`at 10:00 next Monday`

`at 10:00 next month`

The exact accepted date expressions can depend on the implementation.

32. atq

Definition

`atq` lists pending jobs scheduled with `at`.

Syntax

`atq`

Example

`atq`

Example Output

```
5      Thu Sep 24 22:00:00 2026 a john
```

Here 5 is the job number.

33. atrm

Definition

`atrm` removes a scheduled `at` job.

Syntax

`atrm JOB_NUMBER`

Example

`atrm 5`

This removes job number 5.

34. anacron

Definition

`anacron` is used to run periodic jobs such as daily, weekly, or monthly tasks.

Unlike traditional `cron` scheduling, `anacron` is useful when a system may be powered off when a scheduled time occurs.

Syntax

`anacron [OPTIONS]`

Example

`anacron`

Typical Periods

- Daily
- Weekly
- Monthly

Use Cases

- Systems that are not always running.
- Periodic maintenance.
- Backup jobs.
- Cleanup tasks.

35. df

Definition

The df command displays information about disk-space usage of mounted filesystems.

Syntax

df

Example

df

Important Information

- Filesystem
- Total space
- Used space
- Available space
- Usage percentage
- Mount point

36. df -h

Definition

Displays filesystem usage in human-readable units.

Syntax

df -h

Example

df -h

Example output:

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda2	100G	45G	50G	48%	/

37. Essential df Variations

Filesystem type

df -T

Inode usage

df -i

Human-readable + filesystem type

```
df -hT
```

38. dmesg

Definition

dmesg displays messages from the Linux kernel's ring buffer. It is useful for diagnosing hardware, drivers, USB devices, storage, networking, and boot-related issues.

Syntax

```
dmesg
```

Examples

```
dmesg
dmesg | less
dmesg | grep -i error
dmesg | grep -i usb
```

Use Cases

- Hardware troubleshooting.
- Driver troubleshooting.
- USB problems.
- Disk/storage issues.
- Kernel errors.

Note

On some systems, access to kernel messages may be restricted and may require:

```
sudo dmesg
```

39. iostat

Definition

iostat displays CPU and input/output statistics. It is normally provided by the sysstat package.

Syntax

```
iostat
```

Example

```
iostat
```

It can show:

- CPU utilization.
- Device I/O.
- Transfer rates.
- I/O wait.
- Device utilization.

40. iostat 1

Definition

Displays statistics repeatedly at a one-second interval.

Syntax

```
iostat 1
```

Example

```
iostat 1
```

The output is refreshed approximately every second.

Press:

Ctrl+C

to stop it.

41. Essential `iostat` Variations

CPU statistics

```
iostat -c
```

Device statistics

```
iostat -d
```

Extended statistics

```
iostat -x
```

Extended statistics every second

```
iostat -xz 1
```

42. `ip route`

Definition

`ip route` displays and manages the Linux kernel routing table.

Syntax

```
ip route
```

Example

```
ip route
```

Example Output

```
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel scope link src
192.168.1.10
```

Explanation

- default → Default route.
- via → Gateway.
- dev → Network interface.
- src → Preferred source IP address.

43. `ip route | column -t`

Definition

Formats the routing-table output into aligned columns for easier reading.

Syntax

```
ip route | column -t
```

Example

```
ip route | column -t
```

Example formatted output:

```
default          via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0          proto kernel
```

44. Essential ip route Variations

Display routing table

```
ip route show
```

Find route to a specific destination

```
ip route get 8.8.8.8
```

This can show which interface and gateway Linux would use to reach the destination.

45. ss

Definition

ss is used to display socket and network connection information.

It is commonly used as a modern replacement for many netstat use cases.

Syntax

```
ss [OPTIONS]
```

Basic Example

```
ss
```

46. Important ss Variations

TCP sockets

```
ss -t
```

UDP sockets

```
ss -u
```

Listening sockets

```
ss -l
```

All sockets

```
ss -a
```

TCP/UDP listening ports with numeric addresses

```
ss -tuln
```

TCP listening ports

```
ss -ltn
```

UDP listening ports

```
ss -lun
```

Show process information

```
sudo ss -tulpn
```

Options

Option	Meaning
-t	TCP
-u	UDP
-l	Listening
-a	All
-n	Numeric addresses/ports
-p	Process information

47. /proc/cpuinfo

Definition

/proc/cpuinfo is a virtual kernel-provided file containing information about the system's CPU(s).

Command

```
cat /proc/cpuinfo
```

Examples

```
cat /proc/cpuinfo
```

Find CPU model:

```
cat /proc/cpuinfo | grep "model name"
```

Find processor entries:

```
cat /proc/cpuinfo | grep "processor"
```

Use Cases

- Checking CPU model.
- Checking CPU cores/processor entries.
- Checking CPU capabilities.
- Hardware troubleshooting.

48. /proc/meminfo

Definition

/proc/meminfo contains information about system memory provided by the Linux kernel.

Command

```
cat /proc/meminfo
```

Examples

```
cat /proc/meminfo
```

Find memory information:

```
cat /proc/meminfo | grep Mem
```

Find swap information:

```
cat /proc/meminfo | grep Swap
```

Important Entries

Entry	Meaning
-------	---------

MemTotal	Total RAM
MemFree	Currently free memory
MemAvailable	Estimated available memory
Buffers	Buffer memory
Cached	Cache memory
SwapTotal	Total swap
SwapFree	Free swap

49. Linux Log Files

Linux stores system and application logs in locations such as:

`/var/log/`

The exact files vary by distribution and logging configuration.

50. Boot Log

A traditional boot log on some distributions may be:

```
cat /var/log/boot.log
```

Better for browsing

```
less /var/log/boot.log
```

51. chronyd Logs

chronyd is the daemon used by Chrony for time synchronization.

Depending on the Linux distribution/configuration, its logs may be stored in files under:

`/var/log/chrony/`

Check:

```
ls /var/log/chrony/
```

On systemd systems, you can also check:

```
journalctl -u chronyd
```

52. cron Logs

On systems where cron logs are written to a dedicated file:

```
cat /var/log/cron
```

You can also use:

```
less /var/log/cron
```

53. maillog

On systems using the traditional mail log:

```
cat /var/log/maillog
```

Useful for checking mail-related events.

54. secure

On RHEL-family systems, authentication/security-related logs are commonly found in:

```
cat /var/log/secure
```

These can contain information about:

- SSH authentication.
- Login attempts.
- sudo.
- Authentication events.

55. messages

On systems that maintain a traditional messages log:

```
cat /var/log/messages
```

It can contain general system messages.

56. httpd Logs

`/var/log/httpd/` is normally a **directory**, not a file.

List its contents:

```
ls /var/log/httpd/
```

Common Apache log files include:

```
cat /var/log/httpd/access_log
```

and:

```
cat /var/log/httpd/error_log
```

Use:

```
less /var/log/httpd/error_log
```

for easier reading.

57. Useful Log Commands

Instead of `cat`, `less` is often more practical for large log files:

```
less /var/log/messages
```

View the last lines:

```
tail /var/log/messages
```

Follow a log in real time:

```
tail -f /var/log/messages
```

58. Important Distribution Difference

The exact log files depend on the Linux distribution and logging configuration.

RHEL/CentOS/Fedora commonly use files such as:

```
/var/log/messages
```

```
/var/log/secure
```

```
/var/log/cron
```

```
/var/log/maillog
```

```
/var/log/httpd/
```

Debian/Ubuntu commonly use:

```
/var/log/syslog
```

```
/var/log/auth.log
```

Modern systemd systems also provide centralized logs through:

journalctl

59. init [0-6]

Definition

init is the traditional command/interface associated with changing Linux runlevels.

Syntax

`init [RUNLEVEL]`

where the runlevel is generally:

0–6

60. Traditional Runlevels

Runlevel	Traditional Meaning
0	Halt / power off
1	Single-user / rescue mode
2	Multi-user; distribution-dependent
3	Multi-user text mode
4	Unused/custom
5	Graphical mode
6	Reboot

The exact behavior can differ between distributions.

61. init 0

`init 0`

Traditionally shuts down/powers off the system.

Modern systemd equivalent:

`systemctl poweroff`

62. init 1

`init 1`

Traditionally enters single-user/rescue mode.

Modern systemd equivalent:

`systemctl rescue`

63. init 3

`init 3`

Traditionally changes to multi-user text mode.

On a systemd system:

`systemctl isolate multi-user.target`

64. init 5

`init 5`

Traditionally changes to graphical mode.
On a systemd system:
`systemctl isolate graphical.target`

65. init 6

`init 6`
Traditionally reboots the system.
Modern systemd equivalent:
`systemctl reboot`

Important Note

On modern Linux distributions using systemd, init runlevels are largely a legacy interface. `systemctl` targets are preferred.

66. hostnamectl

Definition

`hostnamectl` is used to query and change the system hostname on systemd-based systems.

Syntax

```
hostnamectl
hostnamectl status
hostnamectl set-hostname HOSTNAME
```

Examples

```
hostnamectl
hostnamectl status
Set hostname:
sudo hostnamectl set-hostname server01
Check:
hostnamectl status
```

Use Cases

- Checking hostname.
- Changing hostname.
- System identification.
- Server administration.

67. dmidecode

Definition

`dmidecode` reads information from the system's DMI/SMBIOS tables and displays hardware information.
It is useful for identifying hardware components such as motherboard, BIOS, memory, and processor information.

Syntax

```
sudo dmidecode
sudo dmidecode -t TYPE
```

Examples

Display complete information:

```
sudo dmidecode
```

System information:

```
sudo dmidecode -t system
```

Motherboard information:

```
sudo dmidecode -t baseboard
```

Memory information:

```
sudo dmidecode -t memory
```

Processor information:

```
sudo dmidecode -t processor
```

BIOS information:

```
sudo dmidecode -t bios
```

Use Cases

- Hardware identification.
- BIOS information.
- Motherboard information.
- RAM/slot information.
- Server inventory.

68. arch

Definition

The arch command displays the machine's hardware architecture.

Syntax

```
arch
```

Example

```
arch
```

Example Output

```
x86_64
```

Explanation

Common results include:

```
x86_64
```

```
aarch64
```

Use Cases

- Checking whether the system is 64-bit architecture.
- Selecting compatible software/packages.
- Troubleshooting architecture compatibility.

69. sos

Definition

sos is a diagnostic data collection tool commonly used on Linux systems to collect system information for troubleshooting and support.

Command

On modern versions:

```
sudo sos report
```

Older systems may use:

```
sudo sosreport
```

Example

```
sudo sos report
```

The tool collects relevant system configuration, logs, command output, and diagnostic information into an archive.

Use Cases

- System troubleshooting.
- Support cases.
- Collecting diagnostic information.
- Investigating hardware/software problems.

Important Note

The collected archive may contain sensitive system information. Review handling and sharing requirements before sending a generated report to another person or organization.

70. QUICK REVISION SHEET

System Information

```
date
```

```
uptime
```

```
uname
```

```
uname -a
```

```
uname -r
```

```
uname -m
```

```
cal
```

```
bc
```

```
hostnamectl
```

```
arch
```

```
dmidecode
```

```
sos
```

Services

```
systemctl start <service>
```

```
systemctl stop <service>
```

```
systemctl status <service>
```

```
systemctl restart <service>
```

```
systemctl reload <service>
```

```
systemctl enable <service>
```

```
systemctl disable <service>
```

```
systemctl enable --now <service>
```

Processes

```
ps
```

```
ps -e
ps -ef
ps aux
ps -u <username>
ps -p <PID>
ps -f -p <PID>
```

Real-Time Monitoring

```
top
top -u <username>
Inside top:
c → Full command
k → Kill/send signal
M → Sort by memory
P → Sort by CPU
q → Quit
```

Signals

```
kill -1 <PID>
kill -2 <PID>
kill -9 <PID>
kill -15 <PID>

kill -SIGINT <PID>
kill -SIGKILL <PID>
kill -SIGTERM <PID>
kill -SIGSTOP <PID>
kill -SIGCONT <PID>
```

```
kill -l
```

User Information

```
id
id <username>
```

Job Control

```
jobs
bg %1
fg %1
```

Cron

```
crontab -e
crontab -l
crontab -r
Special schedules:
@reboot
@yearly
@annually
@monthly
@weekly
@daily
```

@midnight

@hourly

One-Time Jobs

at now + 5 minutes

at now + 2 hours

at now + 1 day

at 10:30

atq

atrm <job_number>

Anacron

anacron

Disk

df

df -h

df -T

df -i

df -hT

Kernel

dmesg

dmesg | less

dmesg | grep -i error

dmesg | grep -i usb

I/O

iostat

iostat 1

iostat -c

iostat -d

iostat -x

iostat -xz 1

Routing

ip route

ip route | column -t

ip route show

ip route get 8.8.8.8

Network Connections

ss

ss -t

ss -u

ss -l

ss -a

ss -tuln

ss -ltn

ss -lun

sudo ss -tulpn

CPU

```
cat /proc/cpuinfo
cat /proc/cpuinfo | grep "model name"
cat /proc/cpuinfo | grep "processor"
```

Memory

```
cat /proc/meminfo
cat /proc/meminfo | grep Mem
cat /proc/meminfo | grep Swap
```

Logs

```
cat /var/log/boot.log
journalctl -u chronyd
cat /var/log/cron
cat /var/log/maillog
cat /var/log/secure
cat /var/log/messages
```

```
ls /var/log/httpd/
cat /var/log/httpd/access_log
cat /var/log/httpd/error_log
```

Runlevels

```
init 0
init 1
init 2
init 3
init 4
init 5
init 6
```

Modern equivalents:

```
systemctl poweroff
systemctl rescue
systemctl isolate multi-user.target
systemctl isolate graphical.target
systemctl reboot
```

FINAL COMMAND CHECKLIST

The complete original command list is covered:

```
date
uptime
uname
uname -a
cal
bc
```

```
systemctl start
systemctl stop
systemctl status
```

```
systemctl restart  
systemctl reload
```

```
ps  
ps -e  
ps -ef  
ps aux  
ps -u username
```

```
top  
top -u username  
top → c  
top → k  
top → M  
top → P
```

```
kill -1  
kill -2  
kill -9  
kill -15  
kill -l  
kill -SIGINT  
kill -SIGKILL  
kill -SIGTERM  
kill -SIGSTOP  
kill -SIGCONT
```

```
id  
fg
```

```
crontab  
@reboot  
@monthly  
@weekly  
@daily  
@hourly  
etc.
```

```
at  
atq  
atrm  
at now + ...  
at [time]  
at ... tomorrow  
at ... next Monday  
at ... next month
```

anacron

df

df -h

dmesg

iostat

iostat 1

ip route

ip route | column -t

ss

cat /proc/cpuinfo

cat /proc/meminfo

boot logs

chronyd logs

cron

maillog

secure

messages

httpd logs

init [0-6]

hostnamectl

uname

dmidecode

arch

sos

This version therefore covers the **full command set you originally provided**, while also adding the essential variations needed for practical Linux administration.